

P2401R0

Add a conditional noexcept specification to `std::exchange`

Published Proposal, 2021-06-28

Author:

[Giuseppe D'Angelo](#)

Audience:

SG18, LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

We propose to add a noexcept-specification to `std::exchange`, which is currently lacking one.

Table of Contents

- 1 **Changelog**
- 2 **Motivation and Scope**
 - 2.1 Actual implementations
 - 2.2 What about the Standard Library `noexcept` policy?
 - 2.3 Is this a defect fix?
- 3 **Impact On The Standard**
- 4 **Implementation experience**
- 5 **Technical Specifications**

6 Proposed wording

7 Acknowledgements

References

Informative References

§ 1. Changelog

- R0
 - First submission

§ 2. Motivation and Scope

`std::exchange`'s behavior is fully specified by the Standard in [utility.exchange]:

```
template<class T, class U = T>
constexpr T exchange(T& obj, U&& new_val)
{
    T old_val = std::move(obj);
    obj = std::forward<U>(new_val);
    return old_val;
}
```

The body composes two move constructions (to build `old_val`, and to return it) and an assignment from `new_val` onto `obj`.

Such a specification would allow for a "natural" `noexcept` specification, which is however missing, making `std::exchange` **never** `noexcept` (according to the Standard; but also on multiple implementations, see below).

This is unfortunate; for instance, a primary use case of `std::exchange` is in the implementation of move constructors:

```
class buffer {
    int *data;
public:
```

```

buffer() : data(new int[42]) {}
buffer(buffer&& other) noexcept // at face value, this is noexcept(false)
    : data(std::exchange(other.data, nullptr)) {}
~buffer() { delete [] data; }

~~~
};

```

If one, pedantically, used a conditional `noexcept` specification for `buffer`'s move constructor, using the expressions present in the move constructor itself (i.e. the `std::exchange`), the surprising answer would be that the move constructor in question is *not* `noexcept`.

This can, of course, be generalized to any function that features an usage of `std::exchange` and wants to provide a `noexcept` specification written in terms of its body. While the usage of conditional `noexcept` specifiers is "restricted" in the Standard Library by [P1656]'s policies (but see below), users may want still to use conditional `noexcept` as a contract. In this sense, the Standard Library shouldn't "lie" to them, and claim that operations that cannot possibly throw (by contract of the inner operations) actually may.

Note that all the operations done by `std::exchange`'s body are operations for which a standardized type trait to detect whether the operation is `noexcept` already exists; they are, respectively, `std::is_nothrow_move_constructible` and `std::is_nothrow_assignable`. The very fact that the operations have existing type traits shows that it is meaningful to check for their `noexcept`-ness. A function that composes such operations therefore can very easily carry the information about whether or not it may possibly throw exceptions.

§ 2.1. Actual implementations

[res.on.exception.handling]/5 gives implementations some freedoms to mark functions as `noexcept`, even if they're not specified as such in the Standard:

5: An implementation may strengthen the exception specification for a non-virtual function by adding a non-throwing exception specification.

`std::exchange` does qualify for this. At the time of this writing, [MS-STL] in fact has a conditional `noexcept` specification on `std::exchange` (functionally identical to the one we are proposing), but [libstdc++] and [libc++] do not feature one.

At least in the case of Microsoft's compiler toolchain, the presence of `noexcept` actually improves codegen of `std::exchange` in debug builds.

§ 2.2. What about the Standard Library `noexcept` policy?

[P1656] encodes the latest `noexcept` policy for the Standard Library. Its wording precludes any functions but `swap`, default/copy/move constructors and copy/move assignment operators to be conditionally `noexcept` (points c, d, e).

From a certain point of view, `std::exchange` is extremely similar to `std::swap`: it composes move constructions and a (generalized) assignment. It's hard to see why the two functions should be therefore treated in different ways.

We are therefore asking for a very reasonable exception to [P1656]'s rules in order to make `std::exchange` conditionally `noexcept`.

§ 2.3. Is this a defect fix?

We believe that the change we are proposing should be treated as a defect fix. For this reason we are not proposing a bump in `std::exchange`'s feature-testing macro.

A prior art in this sense is [LWG2762], whose resolution is adding a conditional `noexcept` specification to `unique_ptr::operator*( )`, again in spite of [P1656] strict rules, because of its "obvious" implementation. (Note that the time of this writing, [LWG2762] status is still Tentatively Ready.)

§ 3. Impact On The Standard

This is a pure change for `std::exchange`.

§ 4. Implementation experience

The proposed change has been already implemented and shipped by [MS-STL], and experimentally implemented in [this libstdc++ branch](#) on GitHub.

§ 5. Technical Specifications

All the proposed changes are relative to [N4892].

§ 6. Proposed wording

Modify [utility.syn] as shown:

```
template<class T, class U = T>
    constexpr T exchange(T& obj, U&& new_val)
        noexcept(see below);
```

Modify [utility.exchange] as shown:

```
template<class T, class U = T>
    constexpr T exchange(T& obj, U&& new_val)
        noexcept(see below);
```

Effects: Equivalent to:

```
T old_val = std::move(obj);
obj = std::forward<U>(new_val);
return old_val;
```

Remarks: The exception specification is equivalent to:

is_nothrow_move_constructible_v<T> && is_nothrow_assignable_v<T&, U>

§ 7. Acknowledgements

Thanks to KDAB for supporting this work.

Thanks to Jonathan Wakely for the discussions on the LEWG reflector.

All remaining errors are ours and ours only.

§ References

§ Informative References

[LIBC++]

`std::exchange` implementation in libc++. URL: https://github.com/llvm/llvm-project/blob/699d47472c3f7c5799fe75486689545179cfba03/libcxx/include/__utility/exchange.h

[LIBSTDC++]

[`std::exchange` implementation in libstdc++](https://github.com/gcc-mirror/gcc/blob/9f26e34a5a9614a5b66f146752ecef9ea67b3e2d/libstdc%2B%2B-v3/include/std/utility#L291). URL: <https://github.com/gcc-mirror/gcc/blob/9f26e34a5a9614a5b66f146752ecef9ea67b3e2d/libstdc%2B%2B-v3/include/std/utility#L291>

[LWG2762]

[`unique\\_ptr operator\\*\(\)` should be `noexcept`](https://cplusplus.github.io/LWG/issue2762). URL: <https://cplusplus.github.io/LWG/issue2762>

[MS-STL]

[`std::exchange` implementation in Microsoft STL](https://github.com/microsoft/STL/blob/1866b848f0175c3361a916680a4318e7f0cc5482/stl/inc/utility#L613). URL: <https://github.com/microsoft/STL/blob/1866b848f0175c3361a916680a4318e7f0cc5482/stl/inc/utility#L613>

[N4892]

Thomas Köppe. [Working Draft, Standard for Programming Language C++](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/n4892.pdf). URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/n4892.pdf>

[P1656]

Agustín Bergé. ["Throws: Nothing" should be `noexcept`](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1656r2.html). URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1656r2.html>

[P2401_libstdc++]

[Proof of concept implementation of P2401 in libstdc++](https://github.com/dangelog/gcc/tree/P2401). URL: <https://github.com/dangelog/gcc/tree/P2401>